简单任务对比：效率提升与潜在风险

案例A：标准 ALV 销售订单报表开发验证

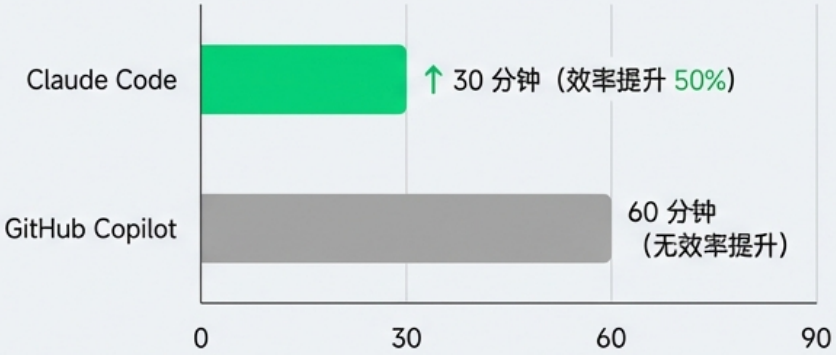
效率两极分化：Claude Code 耗时仅 **30 分钟**，较手写提升 **50%** 效率；GitHub Copilot 耗时 **60 分钟**，无效率提升。

Claude Code 胜出：仅需 **2 轮**人工介入即可运行，而 Copilot 需 **5 轮以上**且伴随运行崩溃（Short Dump）。

关键瓶颈：两者均存在地址取数逻辑错误，且 Claude Code 生成的 SQL 存在 **4 项严重性能隐患**（如 VBFA 大表关联）。

验证结论：在简单逻辑下 AI 表现尚可，但生成的 SQL 缺乏大数据量场景下的性能优化意识。

开发耗时对比（分钟）



核心风险提示：尽管 AI 在简单任务中能显著提升效率，但生成的代码（尤其是 SQL）可能存在严重性能隐患，在处理大数据量场景时需高度警惕，缺乏优化意识。

中等难度挑战：业务幻觉与语法灾难

案例B：采购配额维护（涉及 EQUK/EQUP 表与函数组）

关键问题摘要

核心概念误读：GitHub Copilot 完全混淆业务概念，将“配额”误写为“货源”，导致代码完全不可用。

严重的虚构幻觉：Claude Code 虚构了 10 个不存在的字段（如 BDATV），单函数模块产生 21 个语法错误。

接口规则缺失：AI 无法理解 SAP 函数模块（FM）接口仅接受 DDIC 类型的硬性规则，大量使用全局类型。

修复成本极高：开发人员评估修复 AI 错误的时间已超过重新开发的时间，AI 在此复杂度下失去价值。

错误清单对比表

虚构字段（AI 虚构）	实际字段（DDIC 定义）
⊗ BDATV	✓ EQUK
⊗ MANDT_I	✓ EQUP
⊗ WERKS_I	✓ MANDT
⊗ MATNR_I	✓ WERKS
⊗ EBELN_I	✓ MATNR
⊗ EBELP_I	✓ QUNUM
⊗ QUNUM_I	✓ QUPOS
⊗ QUPOS_I	✓ ERDAT
⊗ ERDAT_I	✓ ERNAM
⊗ ERNAM_I	✓ ZFIELD
⚠ **注意：超过 21 个语法错误和 10 个虚构字段。**	

核心结论：在此复杂度下，AI 开发成本 > 重新开发成本。

高复杂度场景：文档解析与修复陷阱

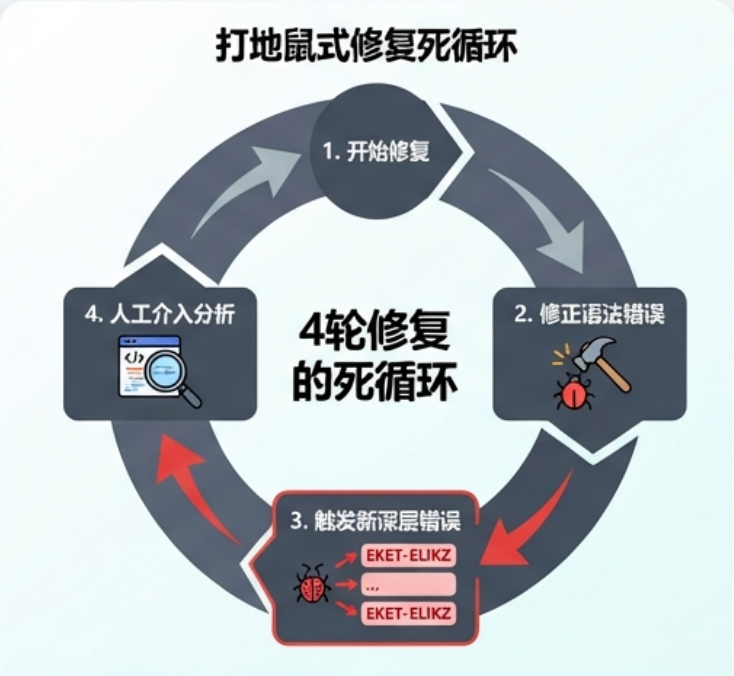
案例C：跨工厂 STO 报表（涉及 18+ 张表与四级追踪）

文档支持差异：Claude Code 支持解析 .docx/.xlsx 需求文档并生成多文件框架，而 Copilot 无法处理。

“打地鼠”式修复：经历 4 轮人工修复，语法错误仍未清零，每修正一个错误即触发更多深层字段错误。

数据字典黑洞：累计出现 10+ 项虚构数据项（如 EKET-ELIKZ），AI 无法感知真实的系统 SE11 环境。

实测评价：结构与字段虚构导致开发人员需读懂上下文并查表替换，修复成本呈指数级增长。



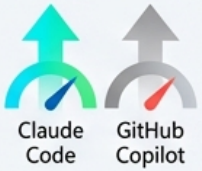
🔑 **核心结论：修复成本指数级增长，开发效率严重受阻。**

结论：AI 编程的“最后一公里”瓶颈

SAP ABAP AI Coding 效率与质量总览

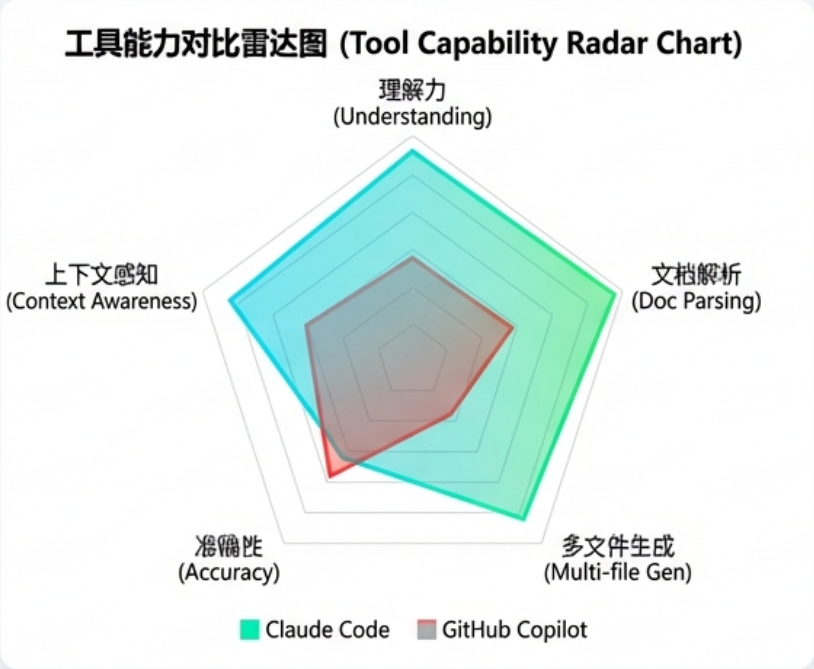
80/20 困局：AI 能生成 80% 的代码框架，但关键的 20%（数据字典准确性）决定了代码能否运行。

核心瓶颈量化：三大案例累计虚构 22+ 个字段，复杂场景下“修复成本 > 直接重写”。



工具性能排位：Claude Code 在理解力、文档解析和多文件生成上全面领先 GitHub Copilot。

未来展望：AI 亟需通过插件或 API 接入 SAP 系统环境（SE11/SE37），消除数据字典“幻觉”是可用性的前提。



核心结论：消除数据字典“幻觉”是实现 AI 编程在 SAP 环境中真实可用的关键前提。